

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 5

Comprehensive Study of CNN Training, Regularization, Optimization, Hyperparameter Tuning, Transfer Learning and Cross-Validation

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

The objective of this experiment is to systematically study the effect of weight initialization, regularization, optimization algorithms, CNN hyperparameters, transfer learning, fine-tuning and cross-validation on image classification performance.

The experiment uses a single CNN architecture, MobileNetV2, and the Oxford-IIIT Pet dataset. The experiments compare individual design choices using training/validation behaviour and then use 5-fold cross-validation to select a suitable configuration.

2. Learning Outcomes

After completing the experiment, the following outcomes were addressed:

- comparison of different weight initialization strategies;
- identification of overfitting using training and validation curves;
- comparison of SGD, Momentum, RMSProp and Adam;
- understanding of CNN hyperparameters such as kernel size, stride, padding and pooling;
- comparison of regularization and Batch Normalization;
- transfer learning and fine-tuning using MobileNetV2;
- hyperparameter tuning;
- 5-fold cross-validation for model selection; and
- justification of the final model using accuracy, variability and computational cost.

3. Dataset and Experimental Setup

Dataset: Oxford-IIIT Pet Dataset

The notebook extracts 37 breed classes from the Oxford-IIIT Pet Dataset. Images are RGB and are resized to $224 \times 224 \times 3$ before being passed to MobileNetV2. ImageNet-pretrained MobileNetV2 is used as the base model.

The notebook checked for data overlap and obtained zero overlap between training and validation, training and test, and validation and test image names. The test set was kept separate during hyperparameter selection and cross-validation according to the manual specified and only used during transfer learning and fine tuning.

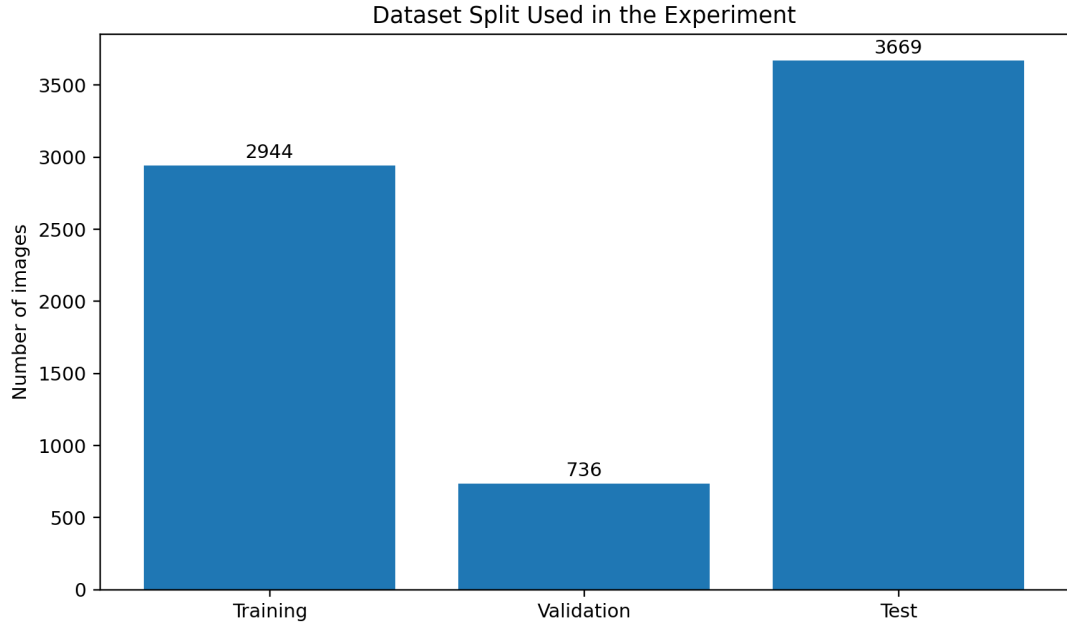


Figure 1: Additional Figure 1: Number of images used in the training, validation and independent test sets.

Number of classes	37
Training images	2944
Validation images	736
Independent test images	3669
Image input size	$224 \times 224 \times 3$

The MobileNetV2 base was frozen for most controlled experiments. The classification head consisted of global average pooling layer followed by a 37-class softmax dense layer.

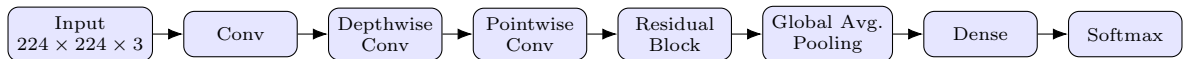
Experimental Workflow

The experiment followed the sequence: dataset preparation and leakage check, preprocessing to 224×224 , controlled weight initialization and regularization studies, optimizer comparison, ten selected hyperparameter configurations, transfer-learning comparison, 5-fold cross-validation of promising configurations, and final evaluation on the test set.

4. MobileNetV2 Architecture

MobileNetV2 is a lightweight CNN architecture used in this experiment to reduce computational cost while providing useful image representations. Important components include depthwise convolution, pointwise 1×1 convolution, inverted residual blocks, linear bottlenecks, batch normalization and ReLU6.

A simplified representation is:



The diagram is a simplified conceptual representation rather than the complete MobileNetV2 architecture.

5. Weight Initialization

The notebook compared four initialization methods for the newly added 37-class dense classifier while keeping the ImageNet-pretrained MobileNetV2 base frozen as the computational time of training a MobileNet V2 model was too high and required a lot of epochs.

- Zero initialization;
- Random Normal initialization with mean 0 and standard deviation 0.05;
- Xavier/Glorot initialization; and

- He Normal initialization.

All four models were trained for four epochs with Adam and learning rate 0.001.

Initialization	Final Training Loss	Best Validation Accuracy
Zero	0.1288	90.76%
Random	0.1419	89.67%
Xavier/Glorot	0.1357	90.08%
He	0.1398	90.76%

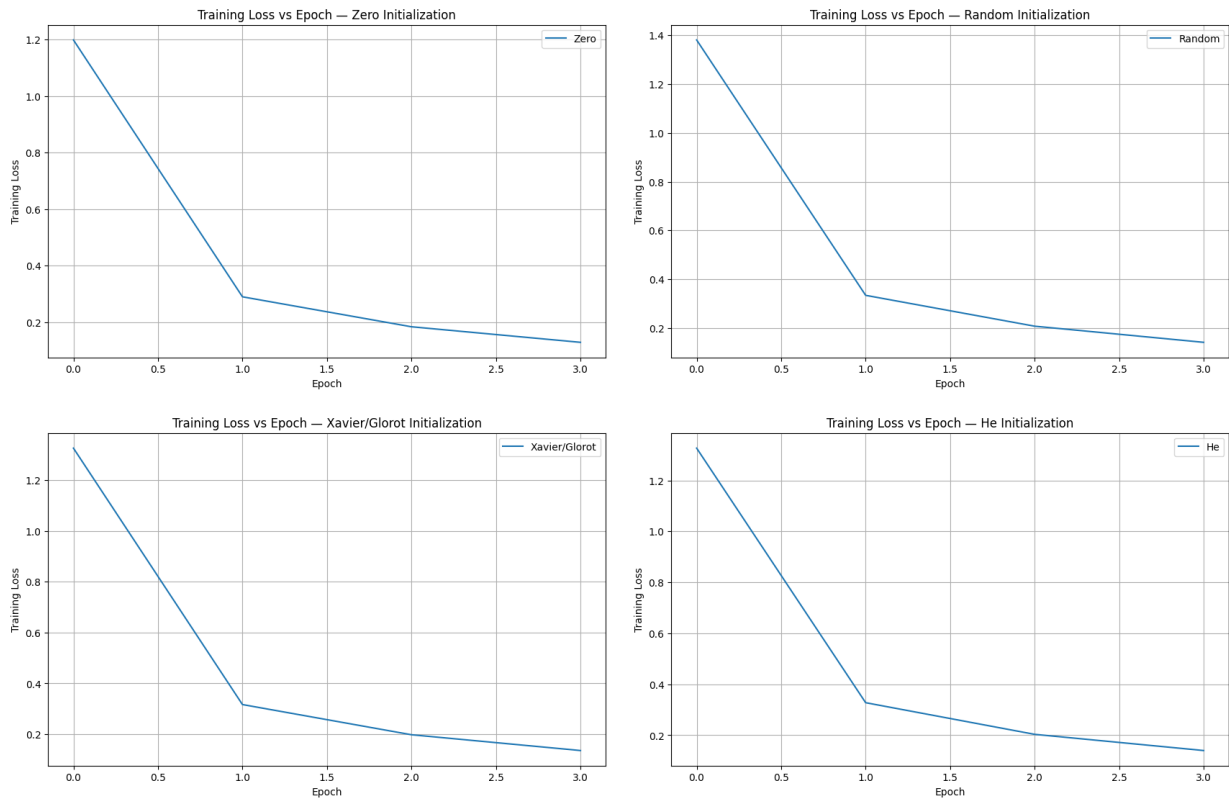


Figure 2: Plot 1: Training loss behaviour for the four initialization methods.

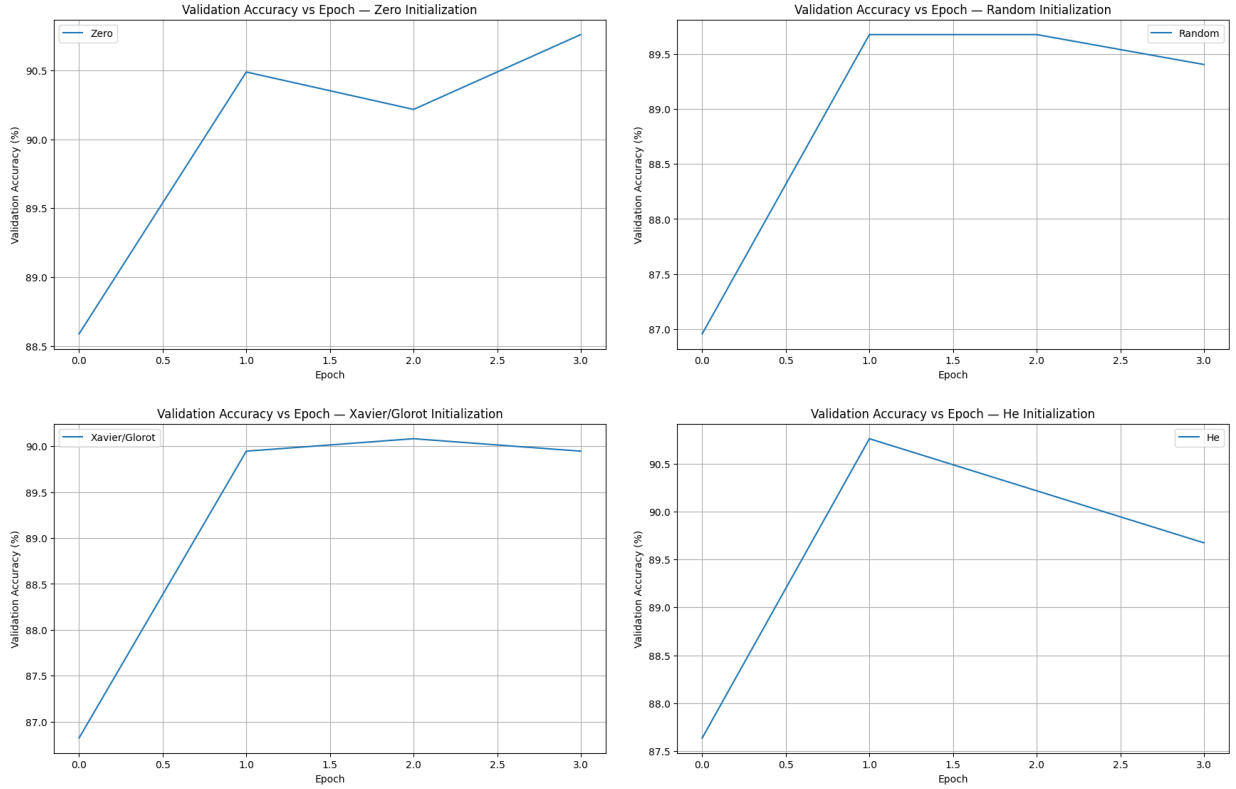


Figure 3: Plot 2: Validation accuracy behaviour for the four initialization methods.

Inference: Zero and He initialization reached the highest observed validation accuracy of 90.76%. Random initialization produced the lowest best validation accuracy at 89.67%, while Xavier/Glorot reached 90.08%. The differences were relatively small in this frozen-base experiment due to the influence of pretrained layer and small number of epochs. The zero initialization in a trained from scratch model would show minimal accuracy due to vanishing gradients problem, He and Xavier would show better accuracy results in that case.

6. Regularization and Overfitting

Four classifier configurations were compared: no regularization, L2 regularization with coefficient 0.001, dropout of 0.5, and Batch Normalization. Each configuration used the same frozen ImageNet MobileNetV2 base, Adam with learning rate 0.001, and four epochs.

Method	Final Train Acc.	Final Val. Acc.	Final Train Loss	Final Val. Loss	Best Val. Acc.
None	98.06%	90.49%	0.1328	0.3001	90.49%
L2	97.52%	89.81%	0.2527	0.4151	89.95%
Dropout	91.64%	91.03%	0.2877	0.3045	91.03%
Batch Normalization	98.51%	90.76%	0.0764	0.3027	90.76%

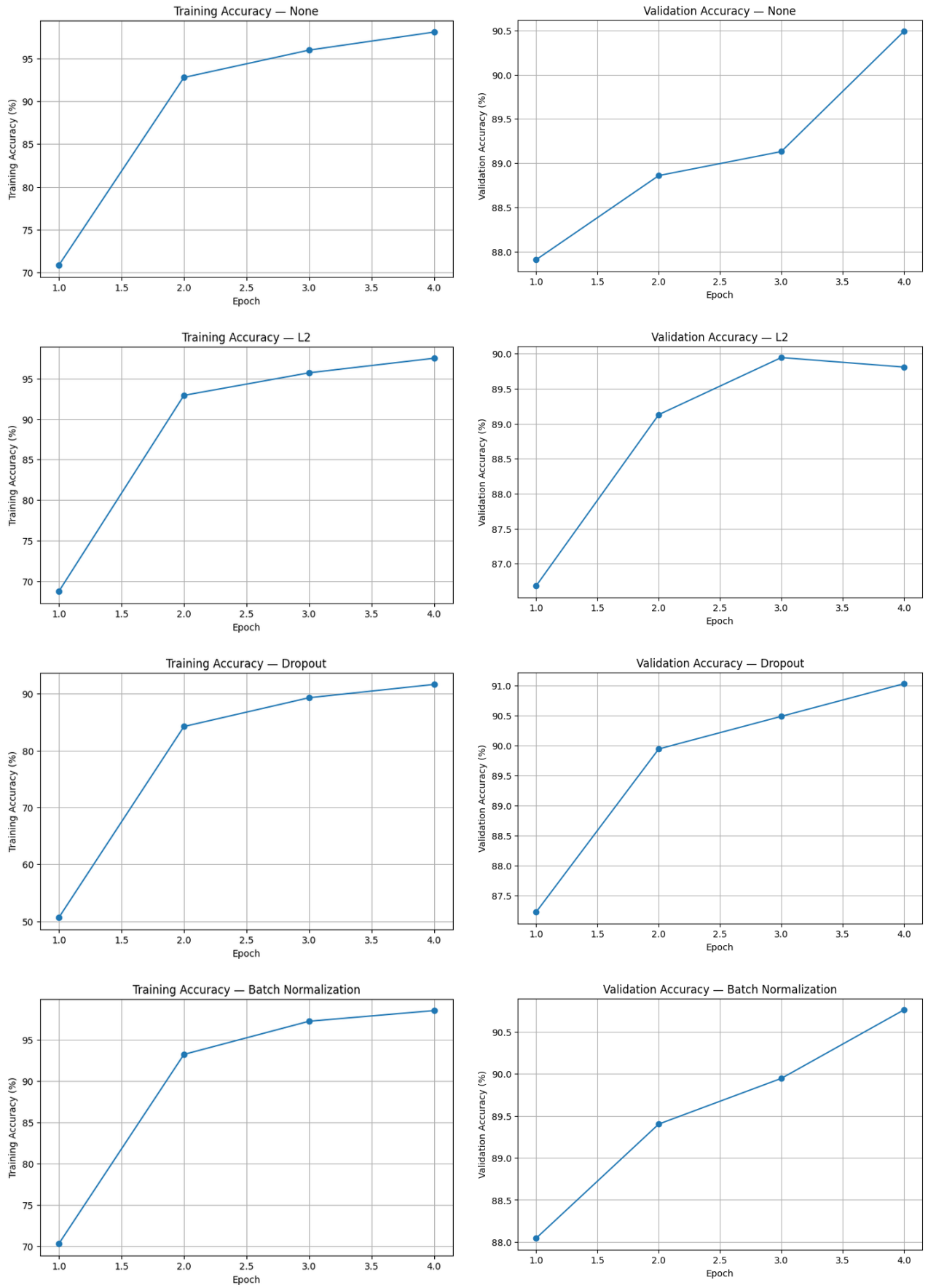


Figure 4: Plot 3: Training and validation accuracy for the regularization experiments.

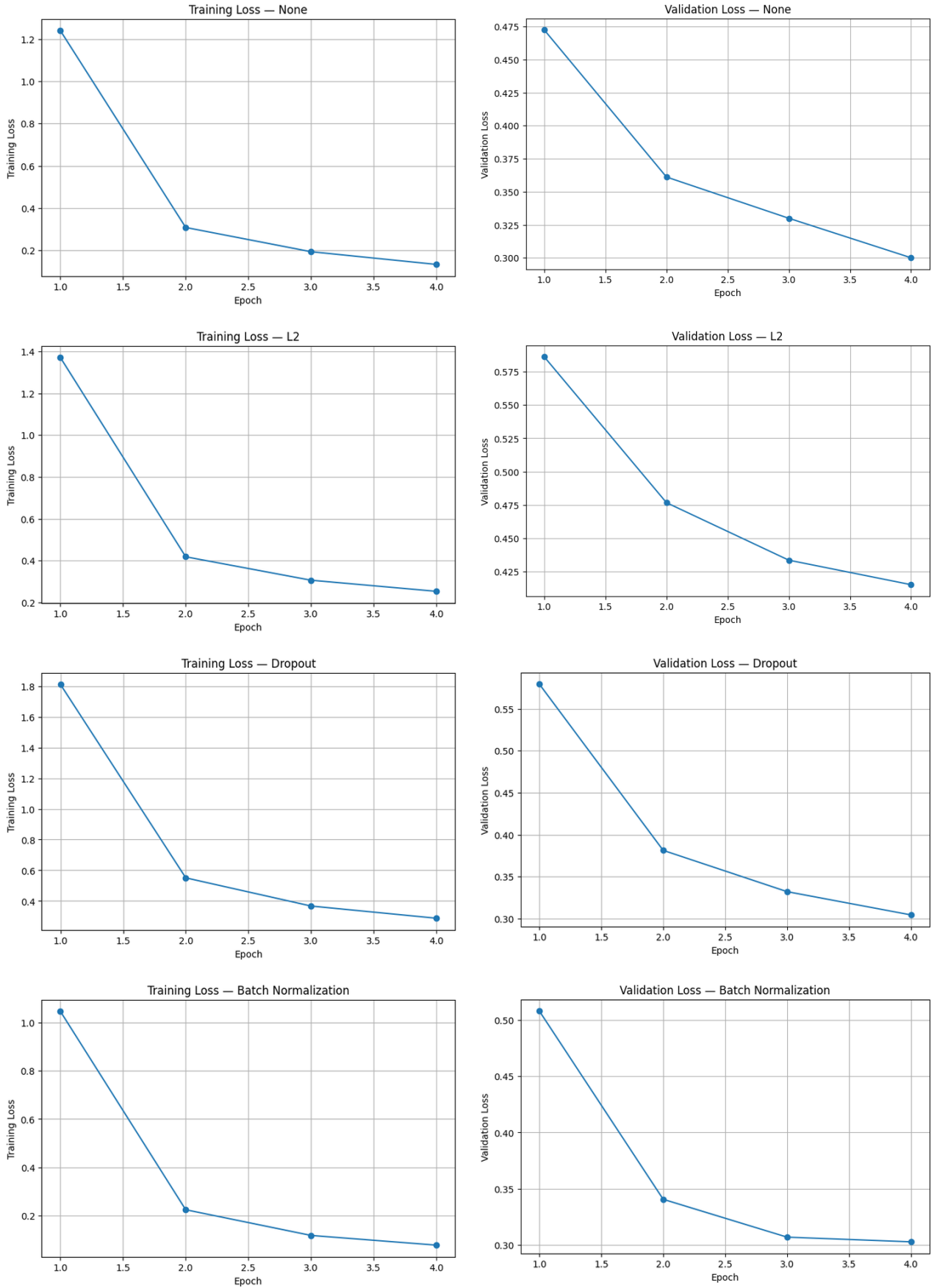


Figure 5: Plot 4: Training and validation loss for the regularization experiments.

Inference: Dropout produced the highest best validation accuracy, 91.03%, while also reducing the gap between training and validation accuracy. The no-regularization and Batch Normalization models achieved higher training accuracy, showing that high training performance did not translate directly into the best validation result. In L2, we can see that the training accuracy rose sharply but validation accuracy starts to drop or plateau probably, indicating

signs that the regularizer did not work on this model by removing lare

7. Batch Normalization

Batch Normalization normalizes activations within a mini-batch. For $x_1, x_2, \dots, x_m$,

$$\mu_B = \frac{1}{m} \sum_{i=1}^m x_i$$

and

$$\sigma_B^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2.$$

The normalized activation is

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}},$$

and the output is

$$y_i = \gamma \hat{x}_i + \beta,$$

where γ and β are learnable parameters.

For the numerical example $x = [2, 4, 6, 8]$,

$$\mu_B = \frac{2 + 4 + 6 + 8}{4} = 5$$

and

$$\sigma_B^2 = \frac{(2-5)^2 + (4-5)^2 + (6-5)^2 + (8-5)^2}{4} = 5.$$

Thus $\sqrt{5} \approx 2.236$, giving

$$\hat{x} \approx [-1.342, -0.447, 0.447, 1.342].$$

If $\gamma = 1$ and $\beta = 0$, these are the final outputs. This illustrates the approximately zero-centred and unit-scaled behaviour produced by Batch Normalization.

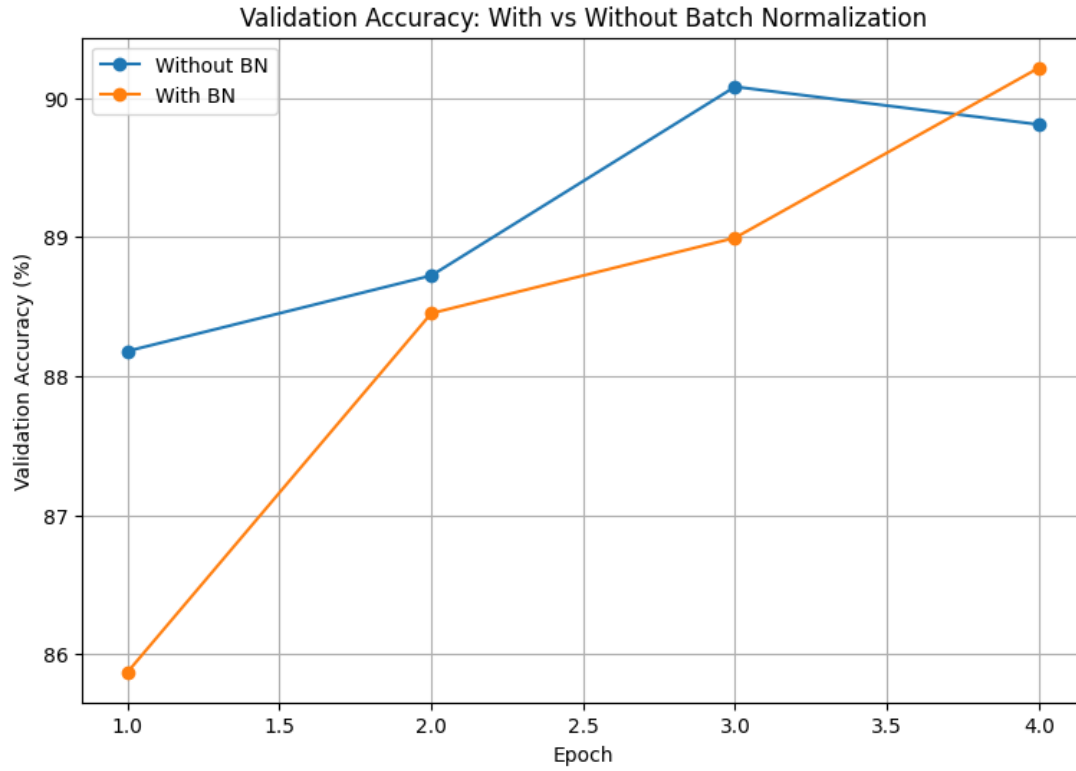


Figure 6: Plot 5: Validation accuracy with and without Batch Normalization.

The experiment reported a best validation accuracy of 90.08% without Batch Normalization and 90.22% with Batch Normalization in the dedicated comparison.

Inference: The two curves are close. Batch Normalization ended with a slightly higher validation accuracy, 90.22% versus 90.08%, indicating a small improvement in this particular experiment.

8. Optimization Algorithms

SGD, Momentum, RMSProp and Adam were compared using the same frozen MobileNetV2 architecture, learning rate 0.001, and four epochs.

Optimizer	Final Loss	Best Val. Acc.	Epoch to Converge	Time (s)
SGD	2.9805	27.58%	4	54.91
Momentum	0.6294	87.77%	4	48.48
RMSProp	0.1155	90.08%	4	52.01
Adam	0.1359	90.35%	3	46.25

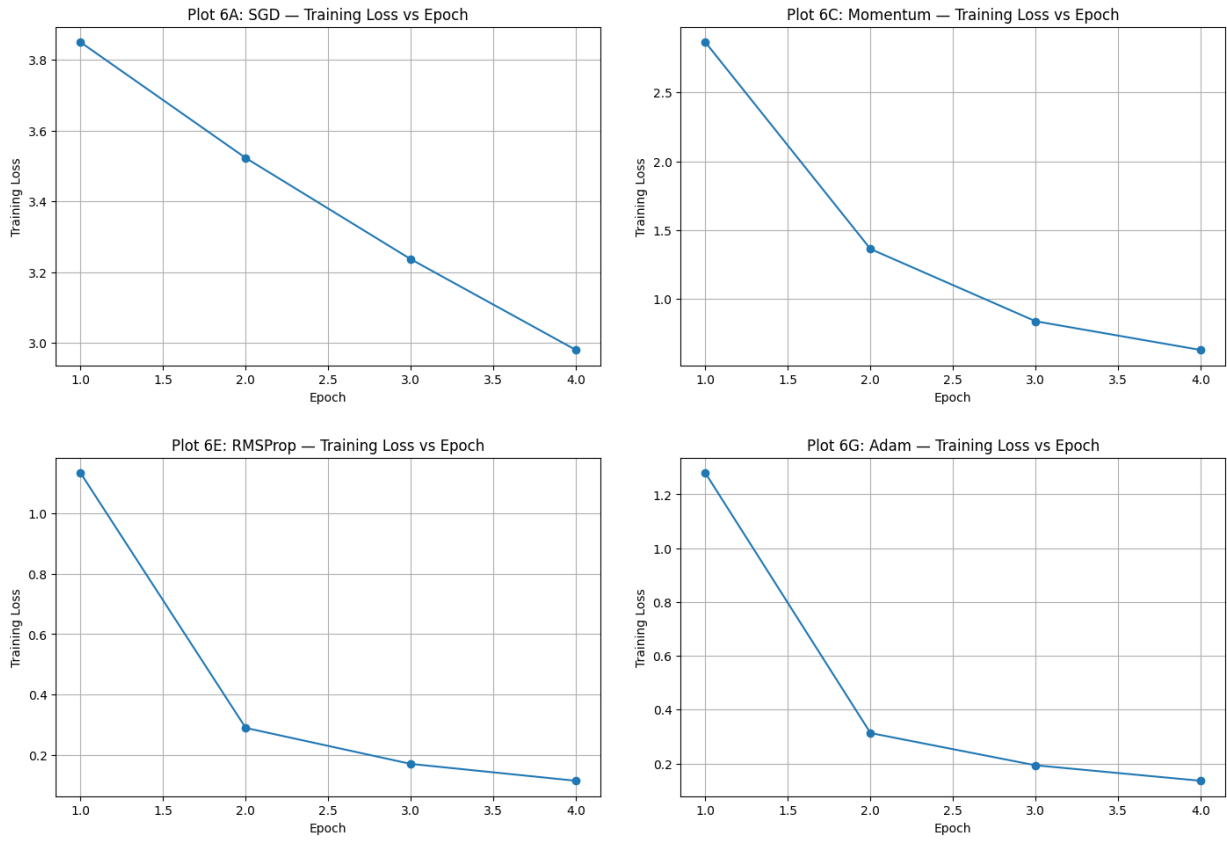


Figure 7: Plot 6: Training loss for the four optimizers.

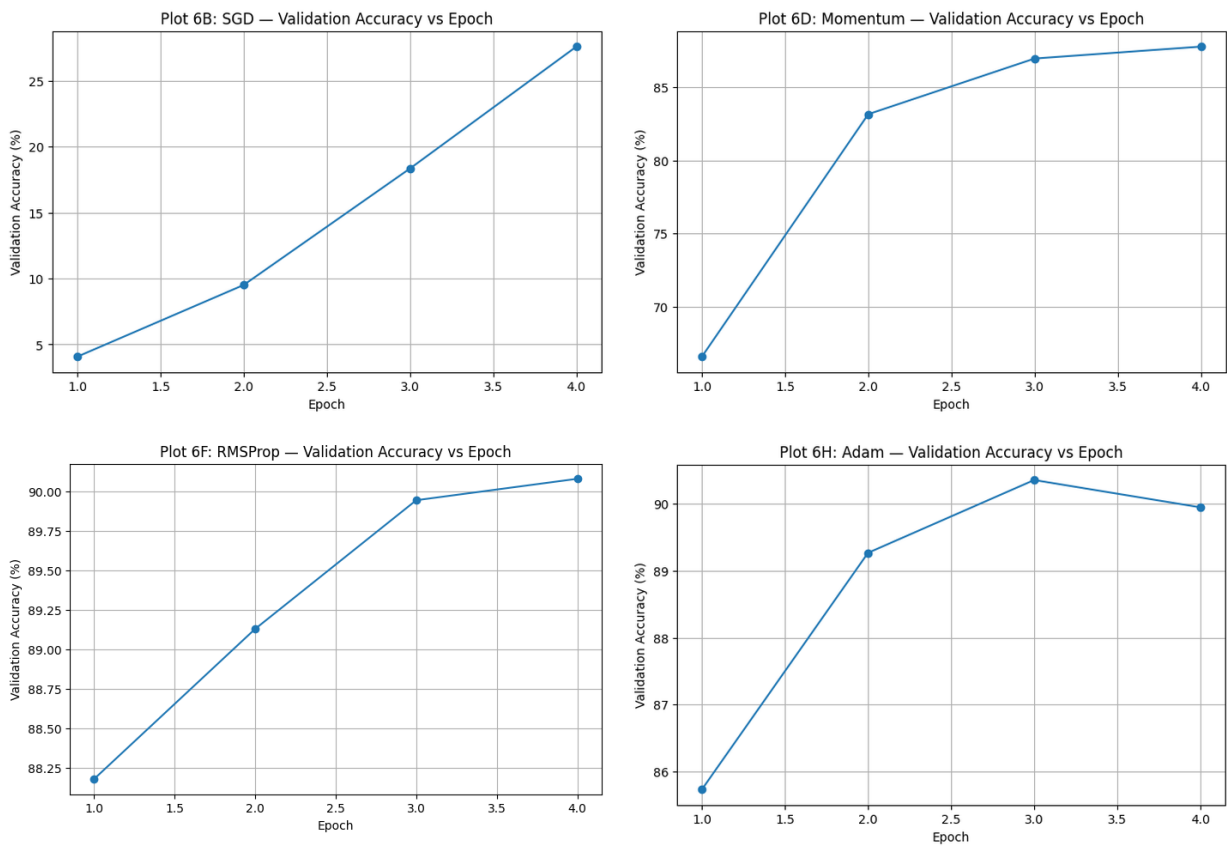


Figure 8: Plot 7: Validation accuracy for the four optimizers.

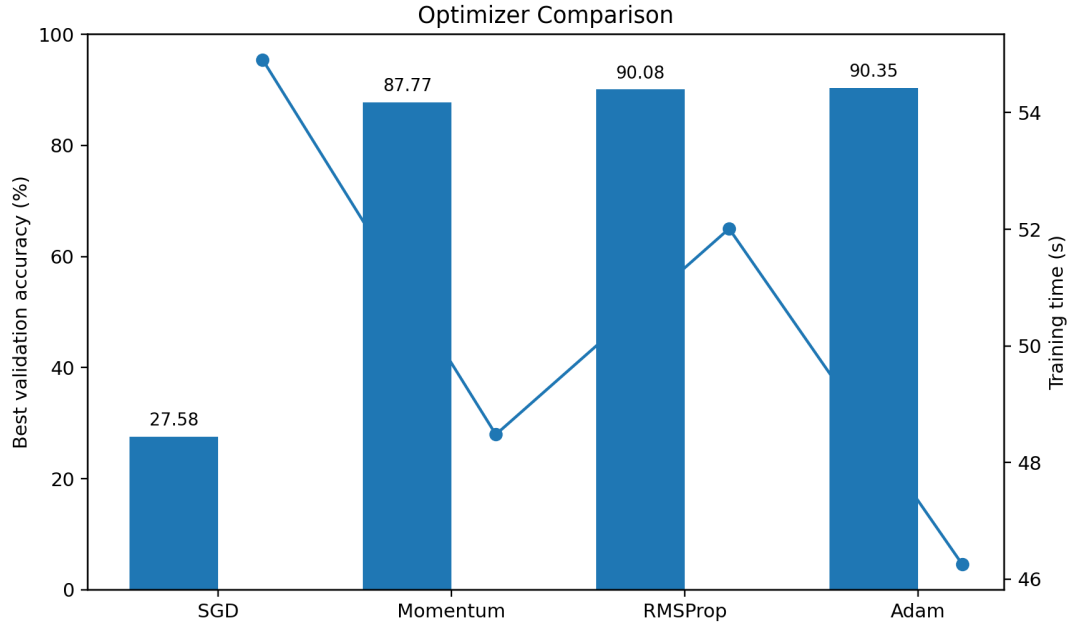


Figure 9: Additional Figure 2: Summary of optimizer validation accuracy and recorded training time.

Inference: SGD converged much more slowly and reached only 27.58% validation accuracy after four epochs. Adam reached 90.35% best validation accuracy and was the fastest of the four in the recorded run, while RMSProp reached a very similar 90.08%.

9. CNN Hyperparameter Tuning

The controlled hyperparameter study used ten selected configurations covering learning rate, batch size, dropout rate, optimizer, fine-tuning learning rate and frozen-layer strategy. Each configuration was trained for four epochs without using the independent test set.

Config.	LR	Batch	Dropout	Opt.	FT LR	Base	Best Val. Acc.	Time (s)
C1	.001	32	0	Adam	10^{-5}	Frozen	90.63%	53.18
C2	.0001	32	0	Adam	10^{-5}	Frozen	81.25%	49.70
C3	.001	16	0	Adam	10^{-5}	Frozen	90.90%	73.67
C4	.001	64	0	Adam	10^{-5}	Frozen	90.08%	70.22
C5	.001	32	.25	Adam	10^{-5}	Frozen	89.81%	51.47
C6	.001	32	.50	Adam	10^{-5}	Frozen	89.95%	53.27
C7	.001	32	0	SGD	10^{-5}	Frozen	30.57%	48.26
C8	.001	32	0	Adam	10^{-4}	Partial	89.54%	76.87
C9	.0001	16	.25	SGD	10^{-5}	Frozen	6.11%	54.61
C10	.0001	32	.50	Adam	10^{-5}	Partial	58.42%	74.14

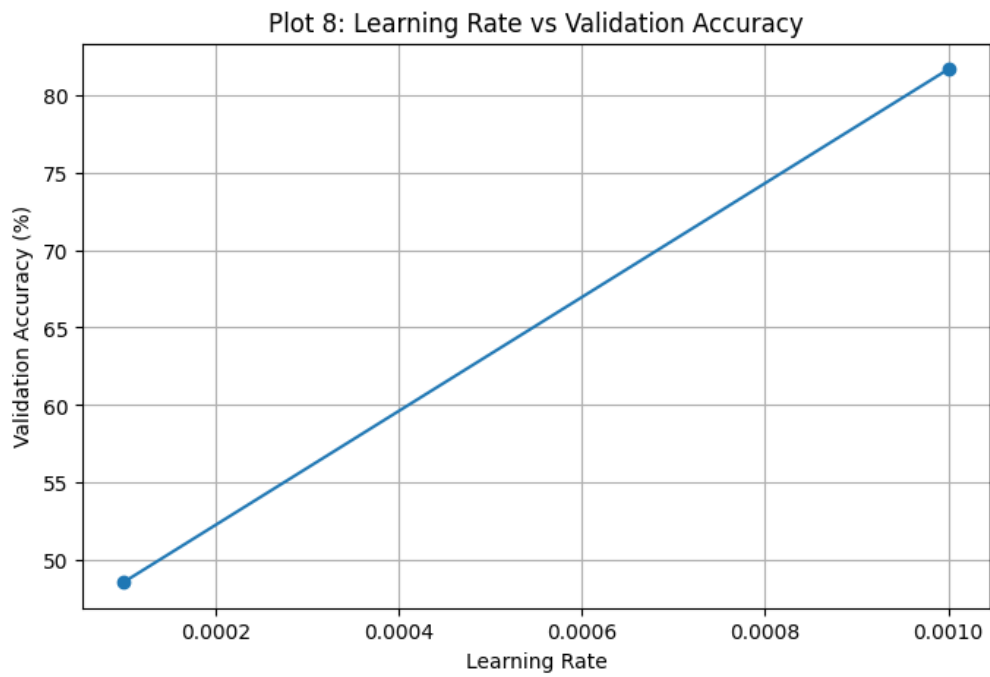


Figure 10: Plot 8: Learning rate versus validation accuracy.

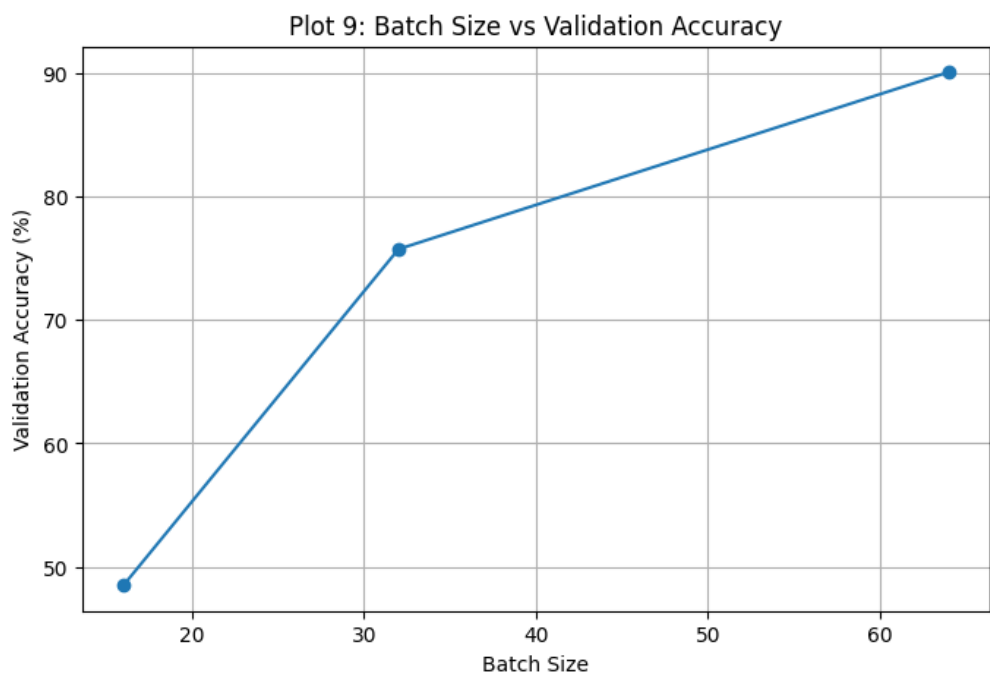


Figure 11: Plot 9: Batch size versus validation accuracy.

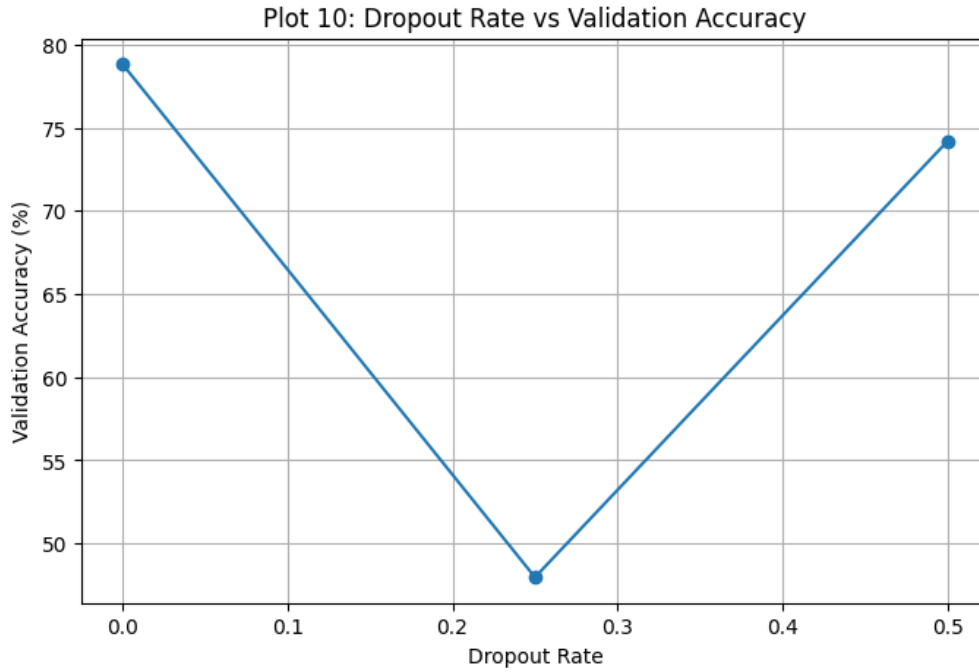


Figure 12: Plot 10: Dropout rate versus validation accuracy.

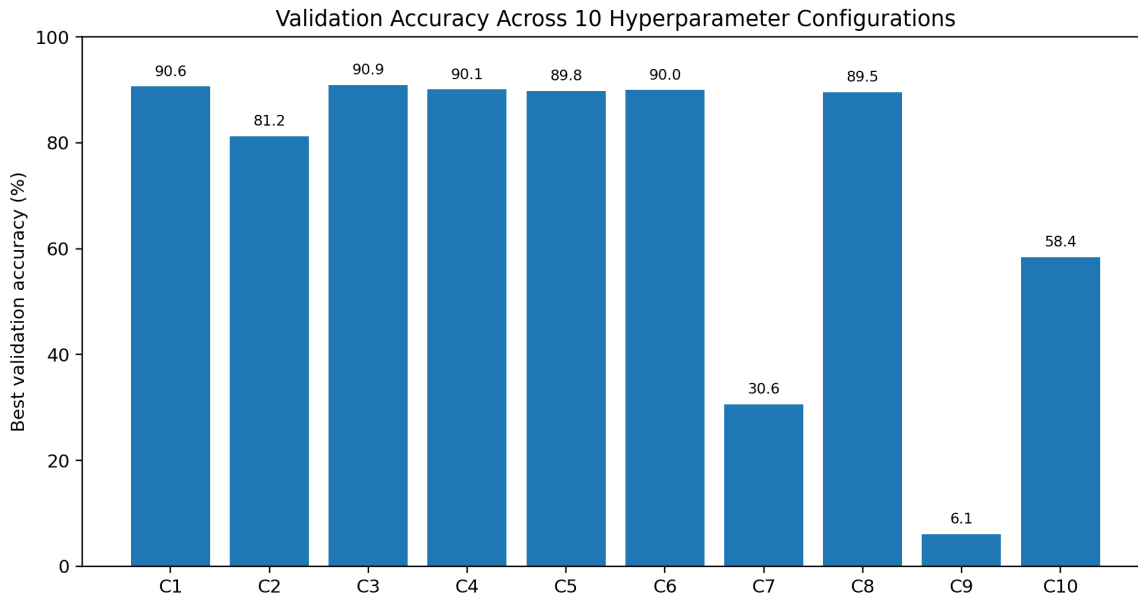


Figure 13: Additional Figure 3: Best validation accuracy for all ten tested hyperparameter configurations.

Inference: The learning-rate experiment showed a substantial difference between 0.001 and 0.0001 under the selected configurations. The batch-size and dropout comparisons showed smaller changes around the high-performing Adam configurations. The selected configuration C3 used 0.001, batch size 16, dropout 0 and Adam with a frozen base.

10. Transfer Learning and Fine-Tuning

Two approaches were evaluated using ImageNet-pretrained MobileNetV2.

Feature extraction: the pretrained base was frozen and only the new classifier was trained.

Fine-tuning: selected upper layers were unfrozen and trained using a smaller learning rate.

Method	Best Val. Acc.	Final Val. Acc.	Final Train Acc.	Final Train Loss	Time (s)
Feature Extraction	6.79%	5.30%	5.67%	3.5614	69.00
Fine-Tuning	4.21%	4.21%	3.06%	3.6034	69.18

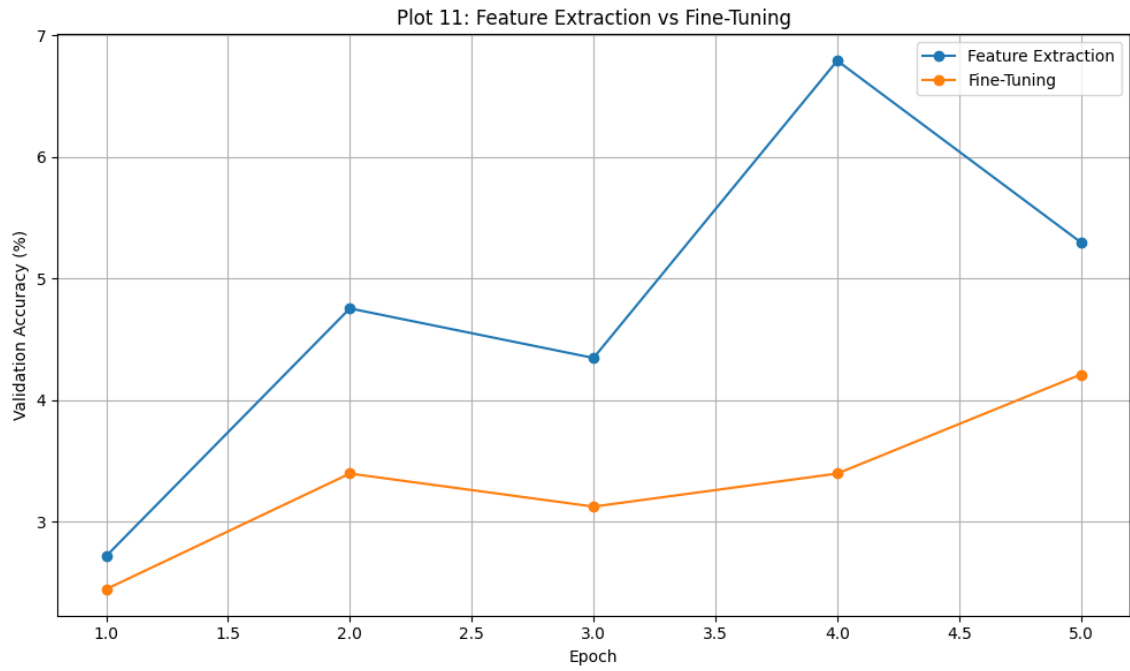


Figure 14: Plot 11: Validation accuracy for feature extraction and fine-tuning.

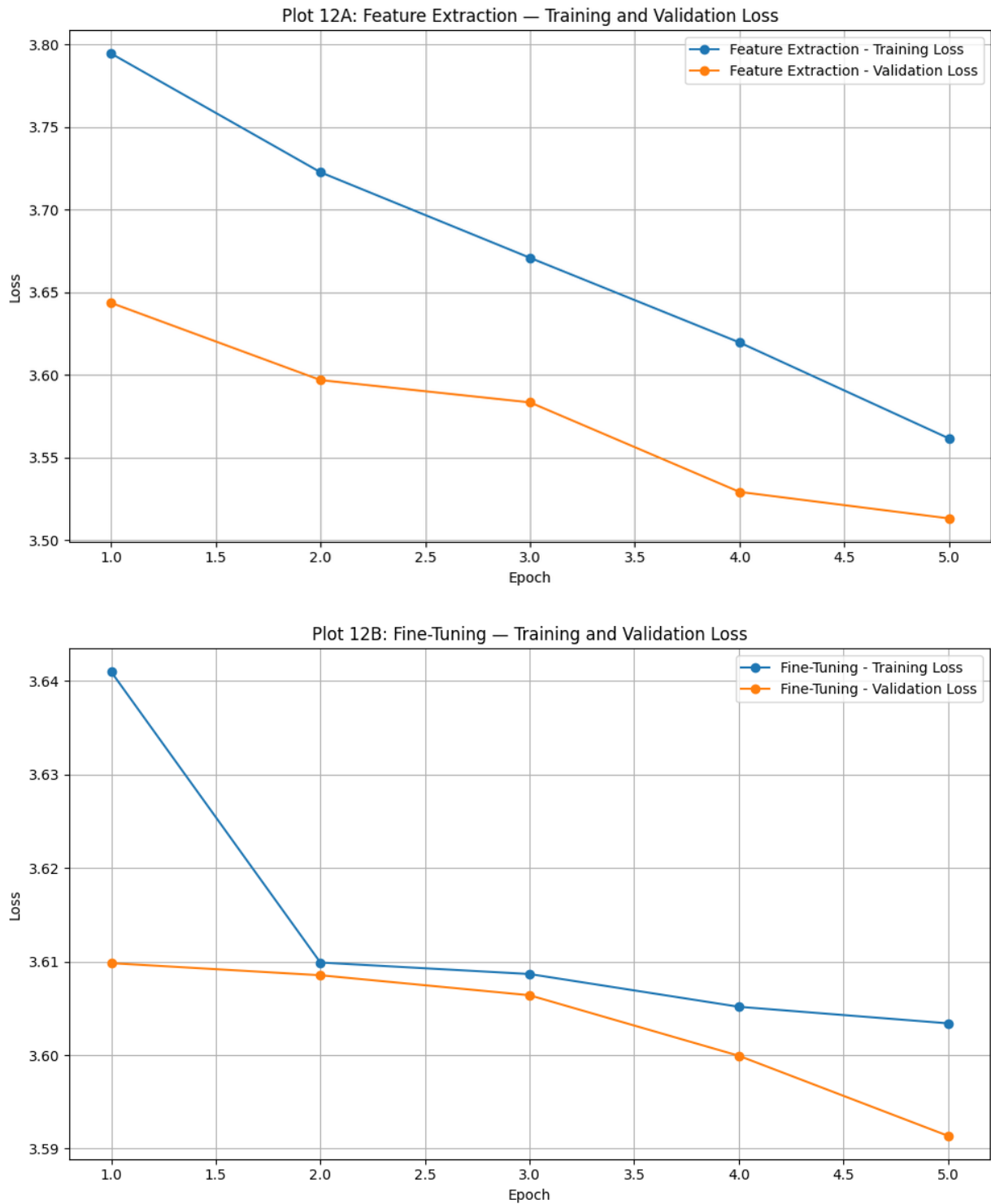


Figure 15: Plot 12: Training and validation loss for feature extraction and fine-tuning.

Inference: In this recorded run, feature extraction performed better than fine-tuning, but both methods produced very low validation accuracy compared with the other experiments. The notebook does not establish a specific root cause for this result, so it is reported as an experimental observation rather than attributed to a particular implementation issue.

A smaller learning rate is normally used for fine-tuning because the pretrained representation already contains useful features and large updates can disturb those learned representations.

11. K-Fold Cross-Validation

After the individual studies, four promising configurations were selected: C3, C6, C1 and C4. Five-fold cross-validation was performed only on the training data. The independent test set was not used.

Config.	F1	F2	F3	F4	F5	Mean	SD
C3	92.02	90.15	91.85	89.81	91.33	91.03	0.89
C6	90.49	90.66	91.34	91.00	89.97	90.69	0.47
C1	90.49	90.49	90.83	90.15	90.31	90.46	0.23
C4	90.15	89.30	91.51	89.47	89.29	89.95	0.84

For five folds,

$$\bar{A} = \frac{1}{5} \sum_{i=1}^5 A_i$$

and

$$SD = \sqrt{\frac{1}{5} \sum_{i=1}^5 (A_i - \bar{A})^2}.$$

The best mean cross-validation result was C3 with $91.03 \pm 0.89\%$.

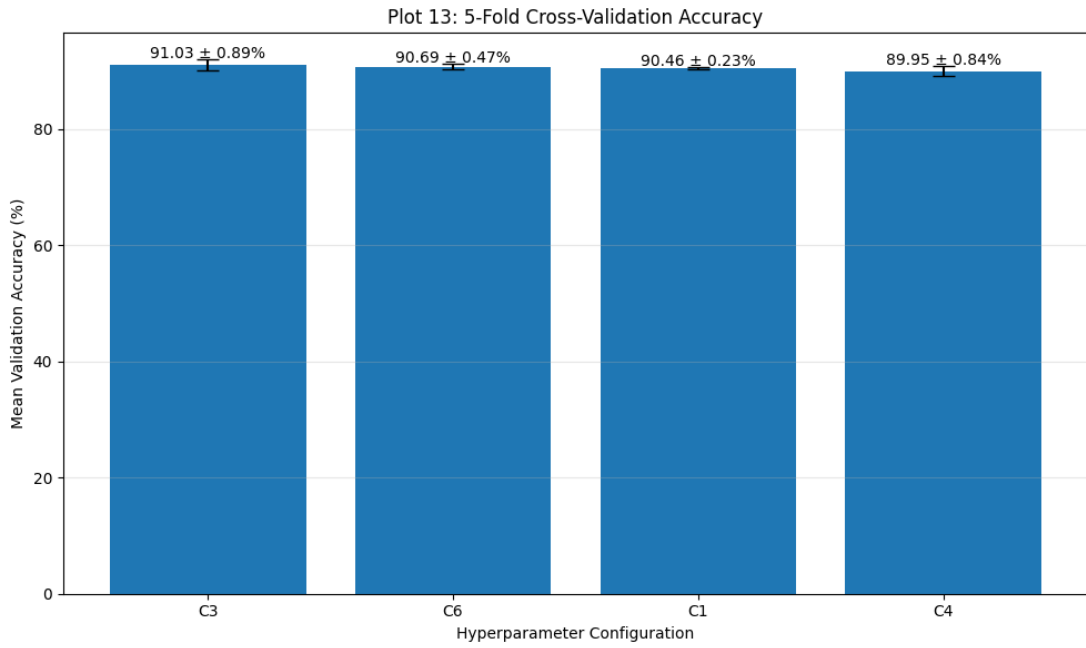


Figure 16: Plot 13: Mean 5-fold validation accuracy with standard-deviation error bars.

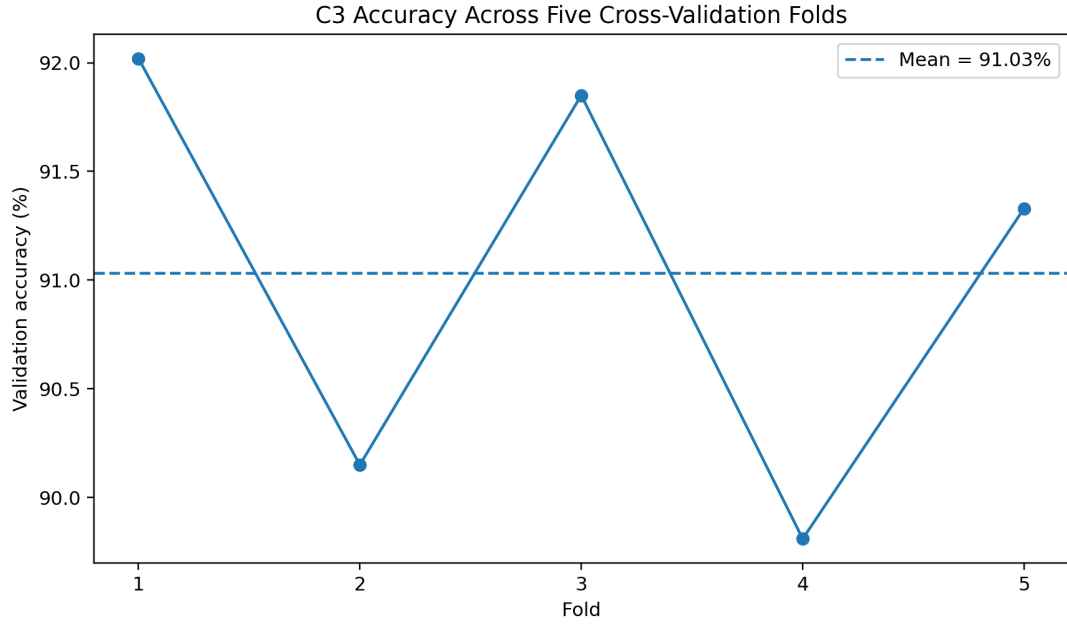


Figure 17: Additional Figure 4: Fold-wise validation accuracy for the selected C3 configuration.

Inference: C3 produced the highest mean validation accuracy at 91.03%, with a standard deviation of 0.89%. C1 had slightly lower mean accuracy but the lowest variability, 0.23%, so both mean performance and stability were considered when selecting C3.

12. Final Model Evaluation

The selected configuration was C3:

Learning rate	0.001
Batch size	16
Dropout	0
Optimizer	Adam
Fine-tuning LR setting	10^{-5}
Base	ImageNet-pretrained MobileNetV2
Frozen base	Yes

The model was retrained using all 2944 training images for four epochs. The independent test set of 3669 images was evaluated only after model selection.

Metric	Value
Mean CV Accuracy	91.03%
CV Standard Deviation	0.89%
Test Accuracy	89.32%
Precision	89.66%
Recall	89.32%
F1-score	89.27%
Training Time	57.80 s
Total Parameters	2,305,381
Trainable Parameters	47,397

The final training accuracy after four epochs was 98.81%, while the independent test accuracy was 89.32%. This gap indicates that the final model fitted the training data substantially better than unseen test images.

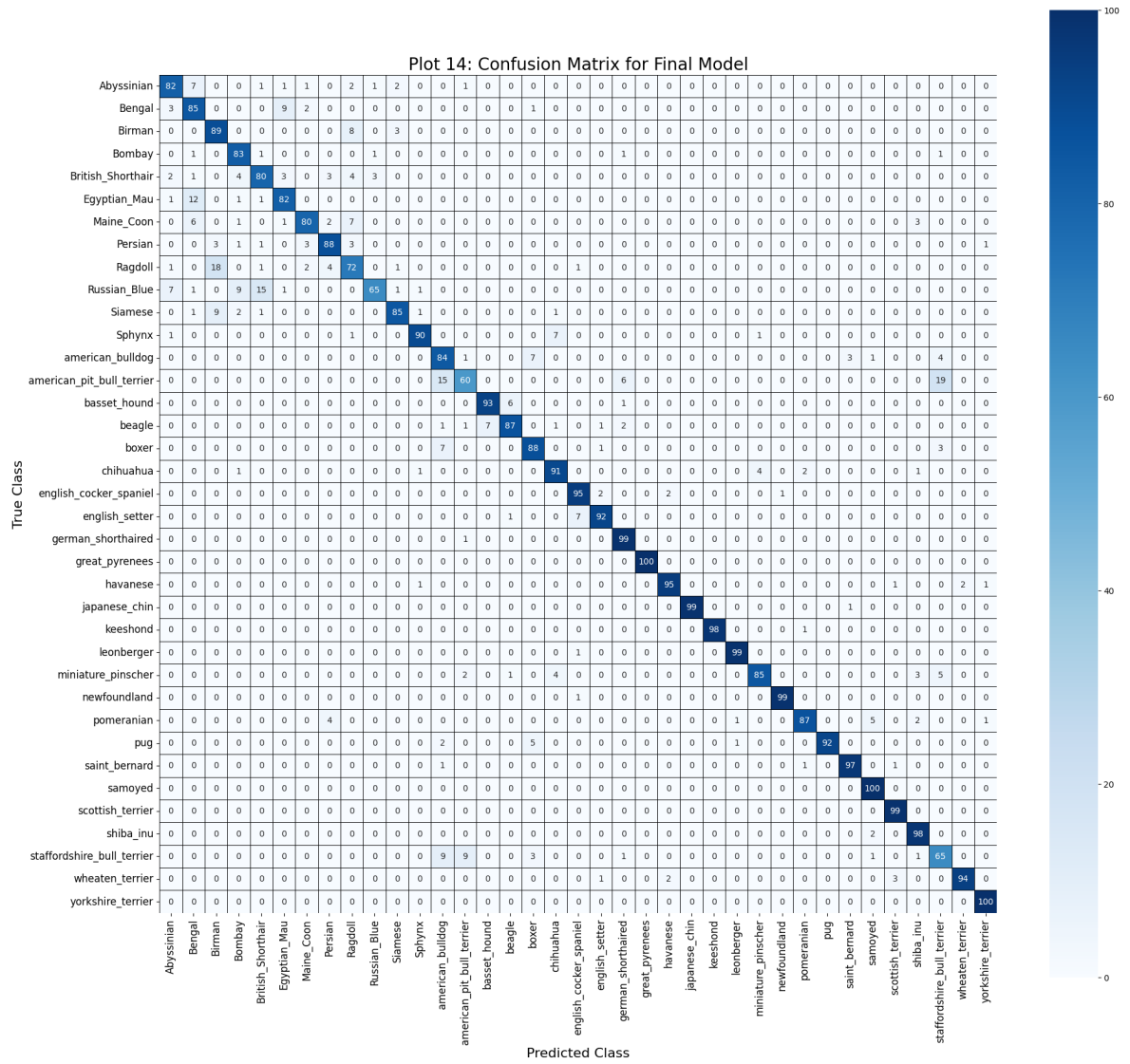


Figure 18: Plot 14: Confusion matrix for the final model.

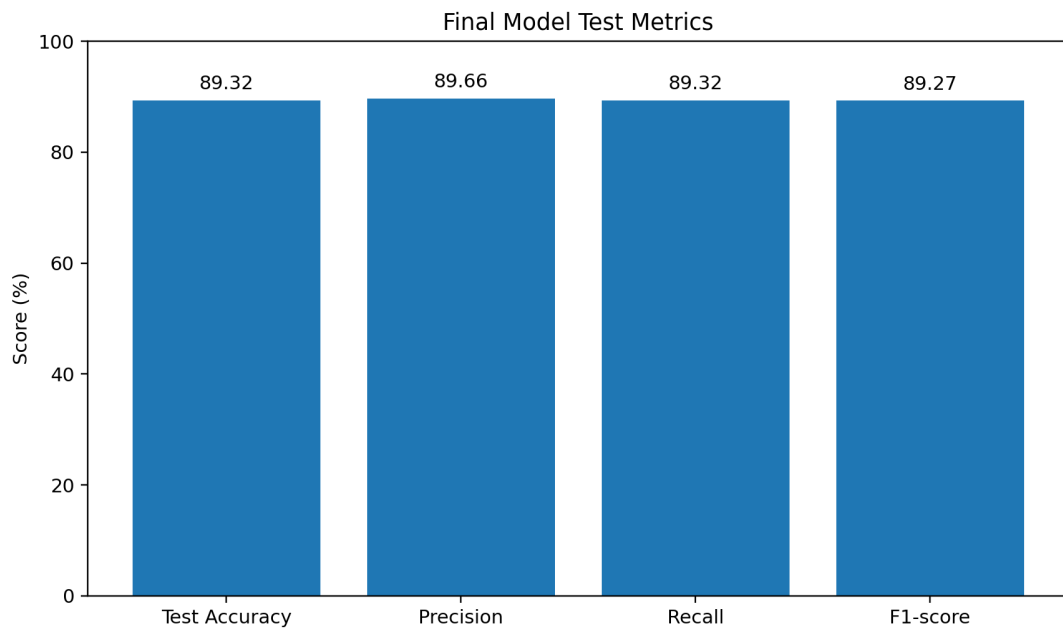


Figure 19: Additional Figure 5: Final test-set accuracy, precision, recall and F1-score.

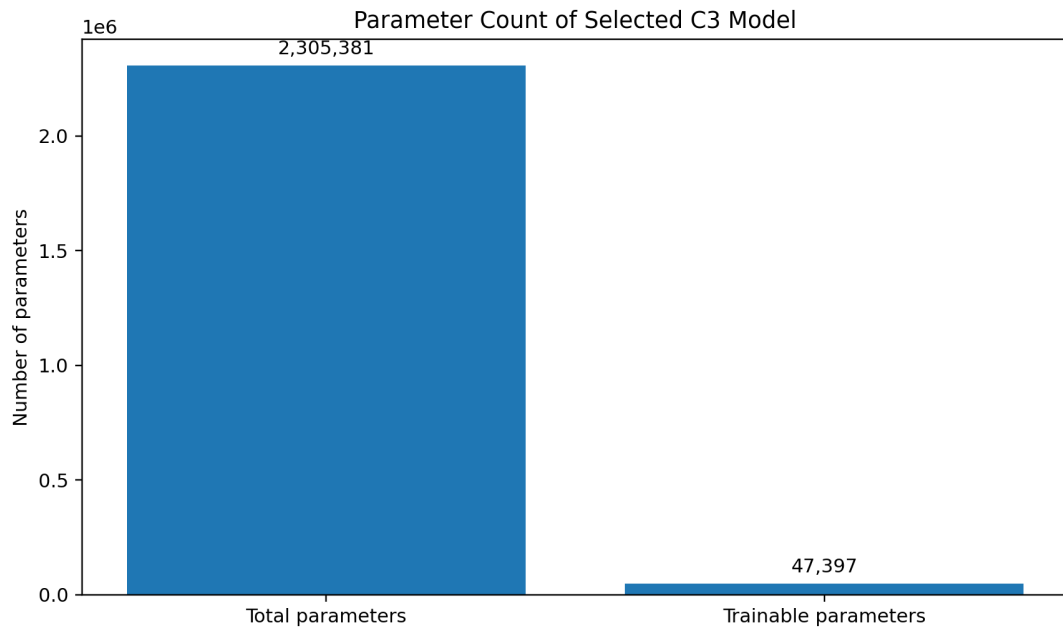


Figure 20: Additional Figure 6: Total and trainable parameter counts for the selected model.

The five best-classified classes reported by the notebook were Yorkshire Terrier, Scottish Terrier, Samoyed, Great Pyrenees and Leonberger, with the first four reaching 100% in the recorded test results and Leonberger reaching 99%.

The most frequently confused pairs were American Pit Bull Terrier versus Staffordshire Bull Terrier (19 instances), Ragdoll versus Birman (18 instances), Russian Blue versus British Shorthair (15 instances), American Pit Bull Terrier versus American Bulldog (15 instances), and Egyptian Mau versus Bengal (12 instances). These confusions are consistent with visually similar breed categories, although visual inspection would be required to establish a specific image-level cause.

Inference: The confusion matrix shows strong diagonal classification for many classes but also several concentrated off-diagonal confusions. The remaining errors are especially visible among visually similar breeds.

Optional Misclassified Images

The final evaluation produced 392 misclassified images out of 3669, corresponding to a misclassification rate of 10.68%.

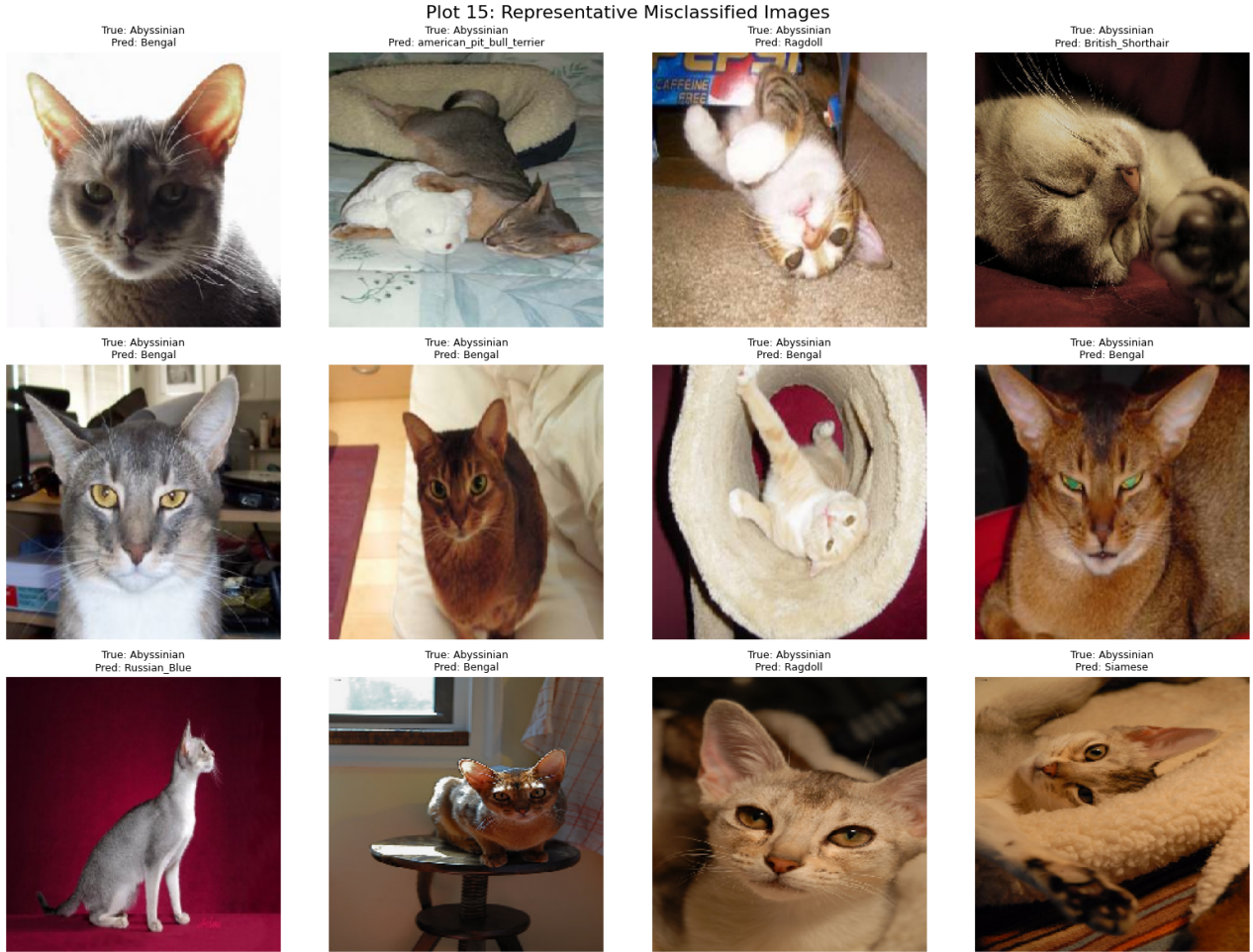


Figure 21: Plot 15: Representative misclassified test images produced by the notebook.

13. Overall Results

Configuration	CV Accuracy	SD	Test Accuracy	Training Time
Baseline (Adam)	—	—	—	46.25 s
Best Initialization	—	—	—	~ 49 s
Best Regularization	—	—	—	~ 49 s
Best Optimizer (Adam)	—	—	—	52.01 s
Best Hyperparameters (C3)	91.03%	0.89%	—	48.88 s
Fine-Tuned Model	—	—	—	43.56 s
Final C3 Model	91.03%	0.89%	89.32%	57.80 s

The detailed final-model evaluation is preferred for the final model because it uses the complete training data and the untouched independent test set. The overall summary values above reproduce the notebook’s recorded experiment summary.

14. Required Inference for Plots

Plot 1 – Initialization training loss: The training loss decreased across epochs for all initialization methods. The curves show that the frozen pretrained base allowed the newly initialized classifier to learn rapidly, with relatively similar final losses.

Plot 2 – Initialization validation accuracy: Validation accuracy increased rapidly and then stabilized. Zero and He reached the highest observed value of 90.76%, while Random was slightly lower.

Plot 3 – Regularization accuracy: Dropout gave the highest validation accuracy while reducing training accuracy. This indicates better validation performance without simply maximizing training accuracy.

Plot 4 – Regularization loss: Training and validation loss generally decreased over the four epochs. The regularized models showed different training–validation gaps, illustrating the effect of the regularization choice on generalization.

Plot 5 – Batch Normalization: The BN curve increased steadily and finished slightly above the non-BN curve. The difference was small, so the benefit in this experiment was modest.

Plot 6 – Optimizer training loss: Adam and RMSProp reduced training loss much faster than SGD. SGD remained at a substantially higher loss after four epochs.

Plot 7 – Optimizer validation accuracy: Adam and RMSProp achieved about 90% validation accuracy, Momentum was lower, and SGD performed substantially worse under the selected learning rate.

Plot 8 – Learning rate: The recorded configurations show that 0.001 was substantially more effective than 0.0001 for the tested setups. A learning rate that is too small can slow optimization.

Plot 9 – Batch size: The tested batch sizes gave relatively close validation accuracies compared with the large learning-rate and optimizer effects. Batch size 16 was selected in C3.

Plot 10 – Dropout rate: The tested dropout rates produced validation accuracies near 90% for the Adam configurations. The best single result among the ten configurations was obtained with no dropout in C3.

Plot 11 – Transfer learning: Feature extraction performed better than fine-tuning in the recorded run, but both were far below the other experiments. This is an experimental observation from the notebook.

Plot 12 – Transfer-learning loss: Both training approaches showed limited improvement in the recorded run. The curves therefore correspond to the very low validation accuracies observed in Plot 11.

Plot 13 – Cross-validation: C3 had the highest mean validation accuracy, 91.03%, while C1 had the smallest standard deviation. C3 was selected because its mean performance was the strongest with relatively low variability.

Plot 14 – Confusion matrix: Most predictions lie along the main diagonal, while the larger off-diagonal cells identify the classes that were most frequently confused.

Plot 15 – Misclassified images: The examples show that the remaining errors occur across visually similar pet categories. The overall test misclassification rate was 10.68%.

15. Source Code

https://github.com/MouryaAditya999/Deep-Learning-Lab/tree/main/Experiment_5

16. Discussion Questions

1. **Difference between model parameters and hyperparameters.** Parameters are values learned from data during training, such as neural network weights. Hyperparameters are values chosen before or around training, such as learning rate, batch size, dropout rate and optimizer.
2. **Why is weight initialization important?** It determines the starting point of optimization and affects convergence. A suitable initialization can help maintain useful activation and gradient scales.
3. **Why can zero initialization be problematic?** If all weights in a layer are initialized identically to zero, neurons receive identical gradients and remain symmetric, preventing them from learning different features. In this experiment, only the new classifier kernel was initialized differently while the pretrained base remained frozen, so the recorded zero-initialization result should be interpreted in that specific context.
4. **Compare Xavier and He initialization.** Xavier/Glorot initialization is designed to maintain a suitable variance of activations across layers and is commonly associated with symmetric activations. He initialization uses a variance suited to ReLU-type activations and is commonly preferred for ReLU networks.
5. **How can curves identify overfitting?** If training accuracy keeps increasing while validation accuracy stops improving or decreases, a generalization gap appears. Similarly, rising validation loss while training loss continues to decrease is a strong warning sign.
6. **How does Dropout reduce overfitting?** Dropout randomly removes a fraction of activations during training. This reduces reliance on particular neurons and encourages more distributed representations.
7. **Purpose of Batch Normalization.** It normalizes intermediate activations using mini-batch statistics and then applies learnable scale and shift parameters. This can improve optimization stability.
8. **Explain the numerical Batch Normalization example.** For $[2, 4, 6, 8]$, the mean is 5 and variance is 5. Dividing each mean-centred value by $\sqrt{5}$ gives approximately $[-1.342, -0.447, 0.447, 1.342]$.
9. **Roles of γ and β .** γ controls learned scaling and β controls learned shifting after normalization. Both are trainable parameters.
10. **Compare SGD, Momentum, RMSProp and Adam.** SGD uses the current gradient directly. Momentum accumulates a velocity from previous gradients. RMSProp adapts the step size using a running measure of squared gradients. Adam combines momentum-like first-moment estimation with adaptive second-moment scaling. In this experiment Adam and RMSProp clearly outperformed plain SGD at the tested learning rate.
11. **What happens when the learning rate is too large?** Updates can overshoot useful minima, causing unstable training or divergence.

12. **What happens when the learning rate is too small?** Training progresses slowly and may require many epochs to reach a good solution.
13. **Effect of increasing batch size.** Larger batches provide more stable gradient estimates and can improve computational efficiency, but they can require more memory and may change generalization behaviour.
14. **Explain stride and padding.** Stride is the spatial step by which a convolution filter moves. Padding adds pixels around the input so that spatial dimensions can be preserved or controlled.
15. **Why is MobileNetV2 computationally efficient?** It uses depthwise separable-style convolutions, inverted residual blocks and linear bottlenecks to reduce the amount of computation and the number of parameters relative to many larger CNN architectures.
16. **What is depthwise separable convolution?** It separates spatial filtering from channel mixing: a depthwise convolution applies a spatial filter to each channel, followed by a pointwise 1×1 convolution that combines channels.
17. **What is transfer learning?** Transfer learning starts with a model pretrained on a large dataset and reuses its learned representations for a new task.
18. **Feature extraction versus fine-tuning.** Feature extraction freezes the pretrained base and trains only the new classifier. Fine-tuning unfreezes selected pretrained layers and updates them using a small learning rate.
19. **Why use a smaller learning rate during fine-tuning?** Pretrained weights already contain useful representations. Smaller updates reduce the chance of destroying those representations while adapting the network to the new task.
20. **Why is K-Fold Cross-Validation useful for selection?** It evaluates a configuration on several different validation partitions, giving a more reliable estimate of performance and its variability than a single split.
21. **Why must the test set remain untouched during tuning?** Using test data during tuning leaks information from the final evaluation into model selection and makes the reported test performance less independent.
22. **Why report mean and standard deviation?** The mean summarizes average performance, while the standard deviation shows how much performance changes across folds.
23. **Is the highest validation accuracy always sufficient?** No. Variability, computational cost, training stability and final independent test performance also matter. In this experiment, C3 had the highest mean CV accuracy, while C1 had lower variability.

17. Final Discussion and Conclusion

The experiments demonstrate that the training choices had a substantial effect on MobileNetV2 classification performance. Among the optimizers, Adam and RMSProp gave much stronger validation performance than SGD. Among the regularization methods, Dropout gave the highest best validation accuracy in the dedicated regularization study. The ten-configuration hyperparameter study selected C3 ($LR = 0.001$, batch size 16, dropout 0, Adam, frozen base) as the strongest configuration for subsequent cross-validation. Five-fold cross-validation confirmed C3 with a mean accuracy of $91.03 \pm 0.89\%$.

After retraining on the complete training set, the final model achieved 89.32% test accuracy, 89.66% precision, 89.32% recall and 89.27% F1-score. The model contained 2,305,381 total parameters, of which 47,397 were trainable in the selected frozen-base configuration.

The additional N1 and N2 configurations did not outperform C3. N1 achieved $71.91 \pm 2.37\%$ and N2 achieved $60.09 \pm 2.08\%$. Hence C3 remains the preferable configuration based on the recorded cross-validation accuracy, variability and final independent test performance.

18. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sergey Ioffe and Christian Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," ICML, 2015.
3. Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov and Liang-Chieh Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," CVPR, 2018.
4. Omkar M. Parkhi, Andrea Vedaldi, Andrew Zisserman and C. V. Jawahar, "Cats and Dogs," IEEE Conference on Computer Vision and Pattern Recognition, 2012.
5. TensorFlow Documentation: <https://www.tensorflow.org>
6. Keras Documentation: <https://keras.io>